PRÁTICAS INTERDISCIPLINARES · ENTREGA 2

SIMSEP — Sprint 1

Evolução das funcionalidades prioritárias
Sistema de Monitoramento e Controle de Segurança em Pedreiras

---

Período da Sprint: 19 a 25 de outubro de 2026

Objetivo: evoluir a primeira versão do sistema, corrigindo os problemas identificados na apresentação anterior e implementando novas histórias prioritárias.

Aluno: Gabriel Henrique Damaceno Alves

Instituição: Universidade Estadual de Goiás — UEG Trindade · Sistemas de Informação

Repositório: `github.com/7121895128/simsep`

1. Visão geral da Sprint

A Sprint 1 desta etapa partiu da versão apresentada na Entrega 1 (sistema com login, pontos de monitoramento, detonações, medições, alertas automáticos, painel administrativo e painel público) e teve como foco três frentes: **(a)** corrigir os problemas apontados e os encontrados em revisão técnica do código; **(b)** implementar as histórias que estavam pendentes ou parciais no Product Backlog (US07, US09 e US10); e **(c)** introduzir testes automatizados, que até então eram apenas manuais.

3
histórias concluídas (US07, US09, US10)

9
correções de problemas identificados

45
testes automatizados (25 unitários + 20 de integração)

38
arquivos alterados · +1.827 / −113 linhas

> **Sprint Goal**
> "Entregar ao Engenheiro de Minas, ao Administrador e à comunidade as visões de histórico, relatório e boletim que faltavam, sobre uma base corrigida e coberta por testes automatizados."

2. Product Backlog atualizado

O backlog foi revisado em 19/10. As histórias concluídas nas Sprints anteriores foram mantidas para rastreabilidade; as marcadas com Sprint 1 foram entregues neste período. Novas histórias e itens técnicos identificados durante a Sprint foram acrescentados ao final, já priorizados.

ID	História de usuário	Prior.	Est.	Status
US01	Como Administrador, quero realizar login no sistema	Alta	3	Concluída
US02	Como Operador, quero cadastrar pontos de monitoramento	Alta	3	Concluída
US03	Como Operador, quero cadastrar detonações programadas	Alta	5	Concluída
US04	Como Operador, quero registrar medições de vibração e risco de desmoronamento	Alta	5	Concluída
US05	Como Operador, quero registrar medições de qualidade do ar e ruído	Alta	5	Concluída
US06	Como Operador, quero alertas automáticos quando os limites forem ultrapassados	Alta	8	Concluída
US07	Como Operador, quero emitir boletins periódicos informativos para a comunidade	Média	5	Concluída Sprint 1
US08	Como Morador, quero um canal público sem login para consultar boletins e alertas	Alta	8	Concluída
US09	Como Engenheiro de Minas, quero consultar o histórico de medições por período, ponto e tipo, com estatísticas e exportação	Média	5	Concluída Sprint 1
US10	Como Administrador, quero visualizar relatórios gerenciais básicos por período	Baixa → Média	8	Concluída Sprint 1
US11	Como Administrador, quero cadastrar usuários pela interface (sem cadastro público)	Alta	3	Concluída
TD01	Como equipe, queremos testes automatizados da API e das regras de negócio para evitar regressões	Alta	5	Concluída Sprint 1

TD02	Como equipe, queremos corrigir as falhas de validação, segurança (XSS) e tratamento de erros encontradas na revisão da Entrega 1	Alta	5	Concluída Sprint 1
US12	Como Administrador, quero editar os limites de segurança (vibração, ruído, ar) pela interface, sem alterar o banco	Alta	3	Pendente (próxima Sprint)
US13	Como Operador, quero receber notificação por e-mail quando um alerta for gerado	Média	5	Pendente
US14	Como Administrador, quero gráficos de evolução no relatório gerencial	Média	5	Pendente
US15	Como Morador, quero acessar o painel público pelo celular com layout adaptado	Média	3	Pendente
US16	Como usuário, quero recuperar minha senha por e-mail	Baixa	5	Pendente

Est. = estimativa em pontos de história (sequência de Fibonacci). A prioridade de US10 subiu de Baixa para Média após a Entrega 1, porque o relatório gerencial é o principal instrumento de acompanhamento do Administrador.

3. Sprint Goal e Sprint Backlog

Capacidade planejada: 23 pontos (US07 5 + US09 5 + US10 8 + TD01 5) mais TD02 (5) absorvido como correções obrigatórias — total de 28 pontos, todos concluídos. A Sprint foi organizada em tarefas técnicas, listadas abaixo com o dia em que foram executadas.

#	Tarefa	História	Dia	Status
T01	Revisão técnica do código da Entrega 1 e registro dos problemas (seção 4)	TD02	19/10	Feita
T02	Migração do banco: `resolvido_em` / `resolvido_por` em alertas, CHECK de valor ≥ 0, índices, tabela `boletins`	US07, US10, TD02	19/10	Feita
T03	Extração das regras de negócio para `services/` (validação, avaliação de limite, CSV, boletim)	TD01	20/10	Feita
T04	Testes unitários com `node:test` (25 casos)	TD01	20/10	Feita
T05	US09 — filtros por ponto/tipo/período, estatísticas e exportação CSV na API e na tela de medições	US09	21–22/10	Feita
T06	US10 — endpoint `/api/relatorios/gerencial` e tela de relatórios (admin)	US10	22–23/10	Feita
T07	US07 — prévia e emissão de boletins; consulta pública e página imprimível	US07	23–24/10	Feita
T08	Correções de segurança e tratamento de erros (XSS, 403, 404 JSON, RBAC por rota)	TD02	24/10	Feita
T09	Testes de integração da API (20 casos) e README atualizado	TD01	25/10	Feita
T10	Sprint Review, Retrospective e consolidação deste documento	—	25/10	Feita

4. Correção dos problemas identificados na primeira apresentação

Dois tipos de problema foram tratados. O primeiro foi o apontado pelo professor na Entrega 1: **a apresentação não trazia os artefatos de engenharia** (desenho do banco de dados, documento de requisitos, diagrama de casos de uso, tipos de testes e histórias de usuário detalhadas). Esses artefatos foram produzidos e compõem a apresentação revisada (Entrega 2 — slides), além de ficarem versionados na pasta `docs/` do repositório. O segundo tipo veio da revisão técnica feita no primeiro dia da Sprint, que encontrou os defeitos abaixo — todos corrigidos e cobertos por teste.

#	Problema identificado	Onde	Correção aplicada	Evidência
P1	Apresentação sem artefatos (requisitos, modelo de dados, casos de uso, testes, histórias detalhadas)	Entrega 1	Artefatos produzidos: documento de requisitos, DER, diagrama de casos de uso, plano de tipos de testes e histórias com critérios de aceitação	Slides da Entrega 2, seções 9–13; `docs/`
P2	**XSS**: nomes de pontos, mensagens e outros dados da API eram inseridos no HTML com `innerHTML` sem escape. Um ponto chamado `<img onerror=…>` executaria script em todos os painéis	`public/js/*.js`	Função `esc()` aplicada em toda renderização; botão “Editar” deixou de serializar o objeto inteiro no atributo `onclick`	Commit 99e1545
P3	Qualquer resposta 403 derrubava a sessão — inclusive o 403 legítimo de “perfil sem permissão”, fazendo o operador ser deslogado ao tocar numa rota administrativa	`menu.js`	`apiFetch` só encerra a sessão em 401 ou em 403 com `Token invalido`	Commit 99e1545
P4	Medição com valor negativo ou tipo inválido chegava ao banco e gerava erro 500 genérico	`routes/medicoes.js`, banco	Validação em `services/validacao.js` (400 com mensagem) + `CHECK (valor >= 0)` no banco	Testes `validacao.test.js`; integração “medicao negativa … 400”
P5	Unidade digitada livremente (“mm/s”, “mm/seg”, “MM/S”) podia divergir da unidade do limite, tornando a comparação enganosa	Tela de medições	Unidade padronizada por tipo e preenchida automaticamente (campo somente leitura)	`UNIDADE_PADRAO` em `validacao.js`

P6	Excluir um ponto com medições/detonações vinculadas retornava 500	`routes/pontos.js`	Responde 409 com mensagem clara; ponto inexistente responde 404	Integração “excluir ponto com medicoes responde 409”
P7	Resolver alerta inexistente retornava 200 com corpo vazio; alerta já resolvido podia ser “resolvido” de novo; não se registrava quem resolveu	`routes/alertas.js`	404 / 409 e gravação de `resolvido_em` e `resolvido_por` (base do indicador de tempo de resolução no relatório)	Integração “resolver alerta grava resolvido_por...”
P8	Controle de acesso por perfil existia só em `/api/usuarios`; um Engenheiro conseguia lançar medições e detonações	Rotas de escrita	`autorizar('operador','admin')` nas rotas de escrita; tela de medições oculta o formulário para o engenheiro	Integração “engenheiro nao lanca medicao (403)”
P9	Rotas `/api/*` inexistentes devolviam HTML do Express; `server.js` não podia ser importado por testes; arquivo `public/.env` com credenciais dentro do pacote	`server.js`, pacote	404 em JSON, `module.exports = app`, rota `/api/saude`, arquivo removido	Integração “rota /api inexistente responde JSON 404”

5. Novas funcionalidades prioritárias implementadas

5.1 US09 — Histórico de medições por período (Engenheiro de Minas)

Critérios de aceitação: filtrar por período (início/fim), ponto e tipo; ver mínimo, média, máximo e quantidade por ponto e tipo; ver limite e situação de cada leitura; exportar o resultado filtrado em CSV abrível no Excel; o Engenheiro apenas consulta, não lança.

Rota	Descrição
`GET /api/medicoes?inicio&fim&ponto_id&tipo`	Histórico filtrado, com `valor_limite`, `situacao` e detonação vinculada (limite de 1.000 linhas)
`GET /api/medicoes/estatisticas`	Mesmos filtros; `COUNT/MIN/AVG/MAX` por ponto e tipo e quantidade acima do limite
`GET /api/medicoes/exportar`	CSV (UTF-8 com BOM, separador “;”, decimal com vírgula) com `Content-Disposition: attachment`

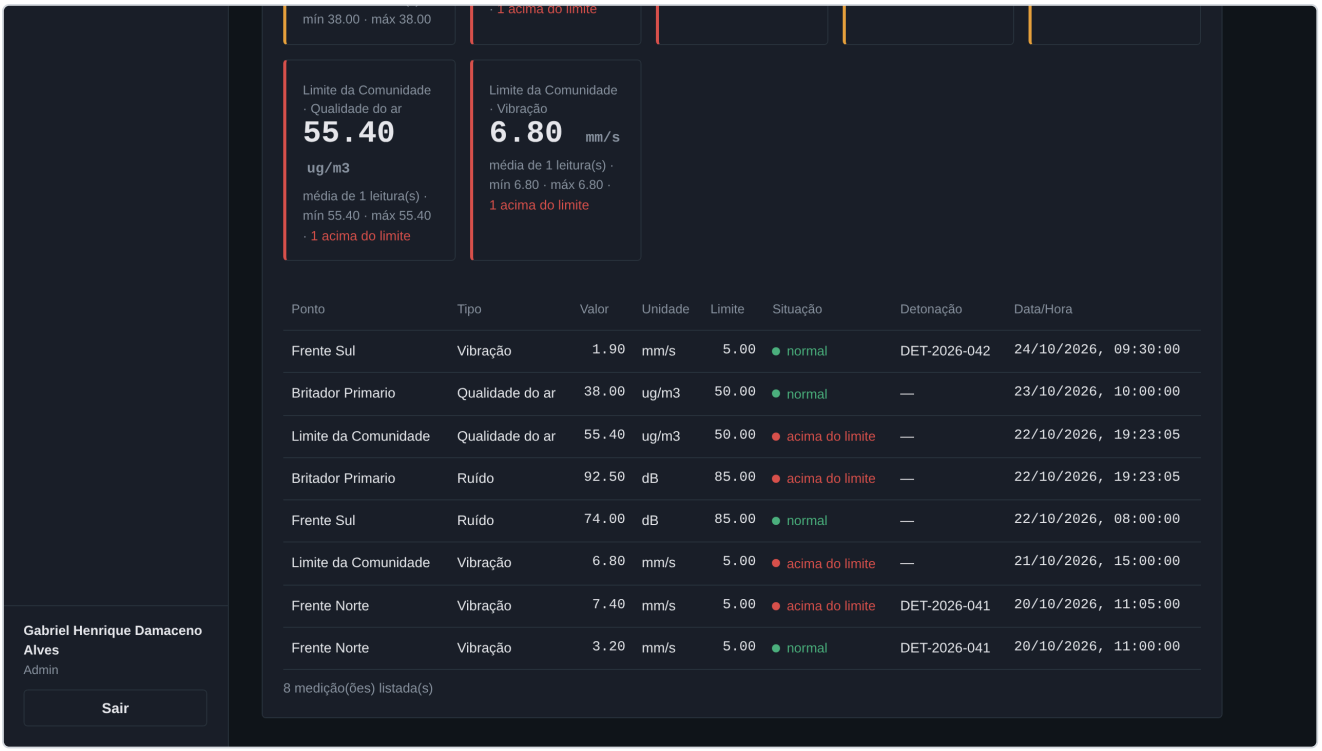


Figura 1 — Tela de medições após a Sprint: filtros por período/ponto/tipo, cartões de estatísticas (vermelho = houve leitura acima do limite), coluna de situação e botão “Exportar CSV”.

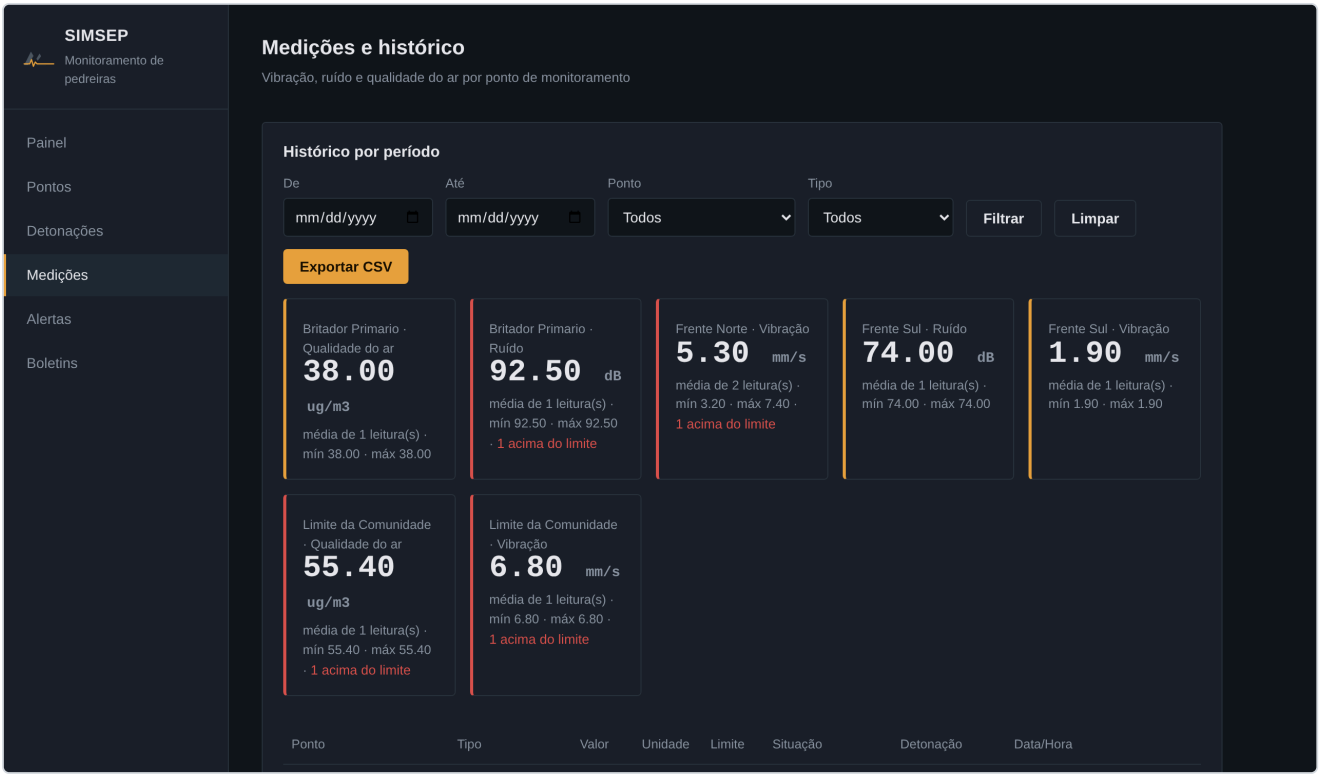


Figura 2 — A mesma tela vista pelo perfil Engenheiro de Minas: o formulário de lançamento não aparece e a API recusa o POST com 403.

5.2 US10 — Relatórios gerenciais básicos (Administrador)

Critérios de aceitação: visível apenas para o administrador; período configurável (padrão: últimos 30 dias); totais de medições, detonações e alertas; percentual de alertas resolvidos e tempo médio de resolução; distribuição por tipo de medição, por ponto e por semana; detonações do período com medições e alertas vinculados; possibilidade de imprimir/salvar em PDF.

Rota	Descrição
`GET /api/relatorios/gerencial?inicio&fim`	Seis consultas agregadas executadas em paralelo (`Promise.all`); protegida por `autorizar('admin')`

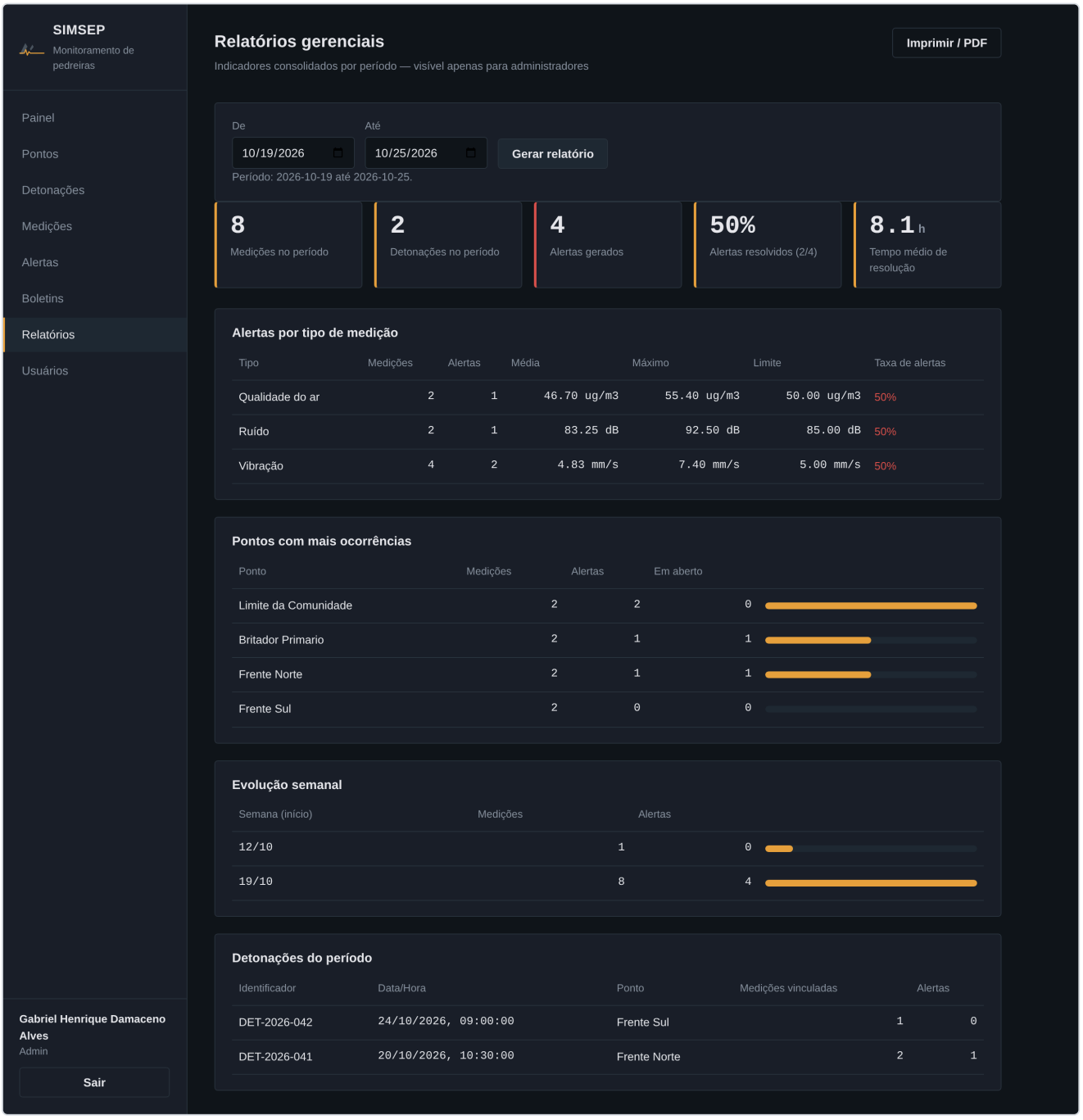


Figura 3 — Relatório gerencial de 19 a 25/10: indicadores, alertas por tipo com taxa, pontos com mais ocorrências, evolução semanal e detonações do período.

5.3 US07 — Boletins periódicos informativos (Operador → Comunidade)

Na Entrega 1 esta história estava **parcial**: o painel público mostrava apenas a situação em tempo real. Agora o operador escolhe um período, pré-visualiza o boletim gerado automaticamente a partir das medições, detonações e alertas, e o emite. A emissão grava um **retrato** dos dados em JSON (`boletins.conteudo`), para que o que a comunidade leu não mude se dados forem corrigidos depois. O boletim fica listado no painel público e abre em uma página própria, imprimível.

Rota	Descrição
`GET /api/boletins/previa?inicio&fim`	Monta o boletim sem gravar (qualquer perfil logado)
`POST /api/boletins`	Emite e publica (operador/admin)
`GET /api/publico/boletins` · `GET /api/publico/boletins/:id`	Consulta pública, sem login, somente leitura

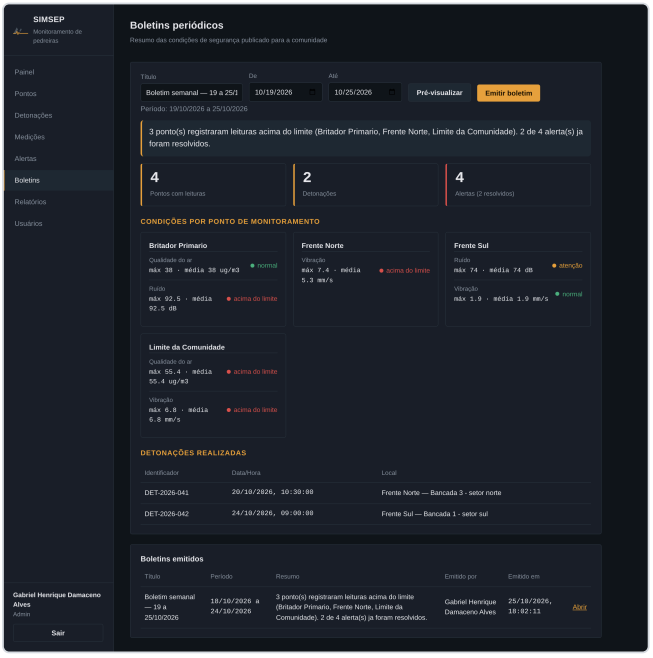


Figura 4 — Tela interna: prévia do boletim semanal antes da emissão.

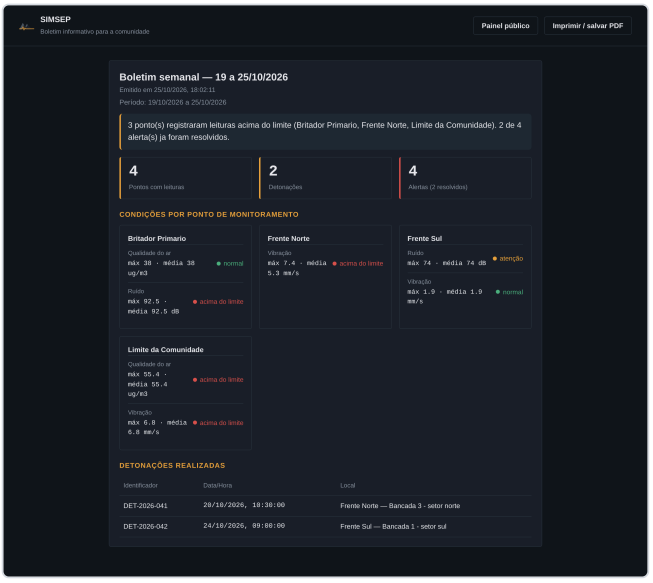


Figura 5 — Boletim publicado, visto pela comunidade (sem login), com botão de impressão/PDF.

5.4 TD01 — Testes automatizados

Foi adotado o `node:test` , executor de testes nativo do Node.js 22, sem novas dependências. As regras de negócio foram isoladas em `services/` para serem testadas sem banco; a API é testada ponta a ponta por HTTP. Detalhes na seção 7.

6. Demonstração do incremento funcional

Roteiro executado na Sprint Review (e reproduzível com os dados de exemplo do repositório):

Passo	Ação	Resultado esperado / observado
1	Login como **operador**; em Medições, lançar vibração 7,4 mm/s na Frente Norte	Mensagem vermelha “Alerta gerado … (+48,0%)”; unidade preenchida automaticamente; medição aparece na tabela como acima do limite
2	Tentar lançar valor −1	Bloqueado pelo navegador (`min=0`) e, via API, HTTP 400 “Valor nao pode ser negativo”
3	Filtrar o histórico de 19 a 25/10 por Frente Norte / Vibração	2 leituras; cartão com média 5,30, mín 3,20, máx 7,40 e “1 acima do limite”
4	Clicar “Exportar CSV” e abrir no Excel	Arquivo `medicoes_2026-10-25.csv` com acentos corretos e colunas Limite/Situação/Detonação
5	Login como **engenheiro**; abrir Medições e Relatórios	Formulário de lançamento oculto; menu “Relatórios” não aparece; acesso direto à URL redireciona ao painel; API responde 403 sem derrubar a sessão
6	Em Alertas, resolver o alerta da Frente Norte; tentar resolver de novo pela API	Status “resolvido … por Operador”; segunda tentativa responde 409
7	Login como **admin**; Relatórios, período 19–25/10	8 medições, 2 detonações, 4 alertas, 50% resolvidos, tempo médio 8,1 h; tabelas por tipo, ponto e semana
8	Boletins: período 19–25/10, “Pré-visualizar” e “Emitir”	Resumo gerado automaticamente; boletim listado em “Boletins emitidos” e no painel público
9	Sem login, abrir `publico.html` e o boletim	Seção “Boletins informativos” com o boletim; página imprimível; nenhum dado de usuário exposto
10	`npm test` e `npm run test:integracao`	25/25 e 20/20 testes aprovados

7. Tipos de testes aplicados

Tipo	O que cobre	Como é executado	Quantidade
Unitário	Regras puras em `services/` : validação de medição/usuário/período, avaliação de limite e mensagem do alerta, classificação de situação, geração/escape de CSV, montagem do boletim	`npm test` (node:test, sem banco, ~0,2 s)	25 casos
Integração / API	Fluxos completos por HTTP contra o servidor e o PostgreSQL: autenticação, autorização por perfil, CRUD, transação medição + alerta, histórico, estatísticas, CSV, relatório, boletim e rotas públicas	`npm run test:integracao` com o servidor no ar	20 casos
Banco de dados	Schema e migração aplicados em banco limpo; constraints (`CHECK` , `FK` , `UNIQUE`) rejeitando dados inválidos; consultas agregadas validadas com massa de exemplo	`psql -f sql/schema.sql` + consultas de verificação	schema + 9 consultas
Funcional manual (aceitação)	Roteiro da seção 6, executado nas telas, cobrindo os critérios de aceitação de cada história	Navegador, perfis operador/engenheiro/admin/público	10 passos

Segurança	XSS em campos de texto, acesso sem token/token adulterado, escalação de perfil, exposição de dados no painel público, `npm audit`	Casos nos testes de integração + revisão manual	6 casos
Regressão	Os cenários da Entrega 1 (401/403/400/409, alerta automático, regras de exclusão de admin) foram transformados em testes automatizados e continuam passando	Incluído na suíte de integração	—

```
$ npm test

# tests 25
# suites 7
# pass 25
# fail 0
# duration_ms 168
```

Saída real do executor de testes na revisão final da Sprint. Os testes de integração exigem o servidor rodando com banco PostgreSQL (instruções no README).

8. Registro da Sprint Review

Data	25/10/2026
Participantes	Gabriel Henrique Damaceno Alves (desenvolvedor e Product Owner do projeto acadêmico); professor orientador da disciplina (avaliador)
Sprint Goal	**Atingido — as três histórias e os dois itens técnicos planejados foram concluídos e demonstrados**
Incremento apresentado	Histórico com filtros, estatísticas e CSV (US09); relatório gerencial (US10); boletins periódicos com publicação para a comunidade (US07); suíte de testes (TD01); nove correções (TD02)
Critérios de aceitação	Todos verificados pelo roteiro da seção 6 e pelos testes automatizados
Itens não concluídos	Nenhum do Sprint Backlog. Durante a Sprint surgiu a necessidade de editar os limites de segurança pela interface (hoje só via SQL) — registrada como US12, primeira prioridade da próxima Sprint
Decisões	(1) Manter o executor nativo `node:test` em vez de Jest/Supertest, evitando dependências; (2) boletim emitido guarda um retrato dos dados (JSONB) em vez de recalcular a cada acesso; (3) Engenheiro de Minas passa a ser perfil somente-leitura; (4) priorizar US12 e US14 (gráficos) na próxima Sprint
Feedback do professor	*(a registrar após a apresentação em sala)*

9. Registro da Sprint Retrospective

O que funcionou bem

- Começar a Sprint com uma revisão técnica do código: os nove problemas da seção 4 foram encontrados em um dia e corrigidos junto com as histórias, sem Sprint dedicada só a bugs.
- Separar as regras de negócio em `services/` tornou os testes unitários triviais e deixou as rotas menores.
- Validar as consultas SQL direto no PostgreSQL com massa de exemplo antes de ligar a tela evitou retrabalho no relatório gerencial.
- Reaproveitar o estilo visual existente: as três telas novas não exigiram CSS novo relevante.

O que pode melhorar

- Os testes de integração dependem de um banco já preparado; falta um script que crie o banco de teste e aplique o schema automaticamente.
- O relatório gerencial ainda é tabular — gráficos facilitariam a leitura pelo administrador (US14).
- Limites de segurança só podem ser alterados por SQL (US12).
- Commits foram agrupados no fim da Sprint; commits diários tornariam o histórico mais rastreável.

Ações para a próxima Sprint

- Criar `npm run db:test` para preparar o banco de teste e rodar a suíte de integração em um comando.
- Implementar US12 (edição de limites) e US14 (gráficos no relatório) como prioridades.
- Commitar ao fim de cada tarefa do Sprint Backlog, referenciando a história no título.
- Manter `npm audit` e `npm test` como checklist antes de cada entrega.

10. Anexos

10.1 Como executar

```
npm install
# banco novo:                  psql -d simsep -f sql/schema.sql
# banco da versão anterior:    psql -d simsep -f sql/migracao_sprint1.sql
cp .env.example .env   # preencher credenciais e JWT_SECRET
npm start              # http://localhost:3000
npm test               # unitários
API_URL=http://localhost:3000 npm run test:integracao
```

10.2 Histórico de commits

Commit	Data	Mensagem
99e1545	26/10/2026	Sprint 1 (19-25/10) — histórico por período, boletins, relatórios gerenciais, testes e correções
1f14195 · 0b1cf5b	26/10/2026	Ajustes visuais do boletim
6fc4116	28/09/2026	Sprint 4 — cadastro de usuários pela tela e correção de vulnerabilidade (base da Entrega 1)

10.3 Estrutura do projeto após a Sprint

```
simsep/
├── server.js                API Express (exporta app para testes)
├── db.js                    pool PostgreSQL
├── middleware/auth.js       autenticar (JWT) e autorizar (perfil)
├── routes/                  auth, pontos, detonacoes, medicoes, alertas, dashboard,
│                            publico, usuarios, relatorios (novo), boletins (novo)
├── services/                validacao, alertas, csv, boletim (novos — regras puras)
├── sql/schema.sql           schema completo · sql/migracao_sprint1.sql (bancos existentes)
├── public/                  telas; novas: relatorios.html, boletins.html, boletim.html
├── tests/unit/              25 testes unitários · tests/integracao/ 20 testes de API
└── docs/                    requisitos, DER, casos de uso, plano de testes, Sprints
```